

HR Analytics: Job Change of Data Scientists

Predicting Candidate Job-Change Likelihood — Project Report

Binary Classification / Model Comparison / Candidate Ranking System

1. Problem Statement

A company runs free data-science training courses. Some trainees stay and take a job with the company (or remain engaged in the field); others use the training only as a stepping stone and are already looking to move elsewhere. Training every candidate is costly, so the company wants to identify — in advance — which candidates are likely to be looking for a new job, so that retention effort and hiring outreach can be focused on the right people instead of applied uniformly.

This is framed as a binary classification problem: given a candidate's demographic, educational, and employment-history attributes, predict whether they will look for a new job (target = 1) or not (target = 0). Beyond a single yes/no prediction, the project also extends the model into a candidate ranking system that surfaces the Top 10 candidates most likely to be job-hunting, since an HR team with limited capacity needs a prioritized shortlist rather than a flat list of flagged rows.

2. Dataset Description

Source: HR Analytics — Job Change of Data Scientists (Kaggle).

Attribute	Value
Training rows	19,158
Training columns	14 (13 features + target)
Test rows (unlabeled)	2,129
Target	1 = looking for a new job, 0 = not looking
Class balance	75.1% class 0 / 24.9% class 1 (imbalanced)

Key data quality issues identified and handled:

- Missing values: up to 32% missing in company_type, company_size, gender, and major_discipline — handled by keeping missingness as its own category rather than imputing it away, since it carries real predictive signal.
- Disguised numeric text: experience ('<1', '>20') and last_new_job ('never', '>4') stored as text — mapped to real integers.
- High-cardinality city column (123 unique values) — dropped in favor of the numeric city_development_index, which captures the same information more efficiently.
- enrollee_id — a unique identifier with no predictive value — dropped.
- Ordinal columns (education_level, company_size) — encoded preserving order; nominal columns (gender, relevent_experience, enrolled_university, major_discipline, company_type) — one-hot encoded.

- Engineered feature: a flag for candidates missing both `company_type` and `company_size`, found to correlate strongly with job-hunting likelihood (candidates missing both fields showed a 2x higher target rate than those with company info present).

3. Methodology

3.1 Preprocessing Pipeline

Raw data was cleaned and encoded per the issues above, producing a fully numeric feature matrix with zero missing values. Features were split 80/20 into training and validation sets using a stratified split to preserve the 75/25 class balance in both partitions.

3.2 Models Trained

Five classification algorithms were trained and compared: Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Random Forest, and XGBoost. Distance-based models (Logistic Regression, KNN, SVM) were trained on standardized (scaled) features; tree-based models (Random Forest, XGBoost) were trained on the raw feature values.

3.3 Handling Class Imbalance

Since the target is imbalanced (~75/25), `class_weight='balanced'` was applied to Logistic Regression, SVM, and Random Forest, and `scale_pos_weight` was applied to XGBoost, so the minority class (job-seekers) is not ignored by the models.

3.4 Threshold Tuning

Rather than using the default 0.5 decision threshold, every model was evaluated across 19 thresholds (0.05 to 0.95) and the threshold maximizing F1-score was selected per model. This matters because accuracy alone is misleading on imbalanced data — a model predicting the majority class for every row would still score roughly 75% accuracy while providing no real value.

3.5 Evaluation Metric

F1-score was used as the primary evaluation metric (the harmonic mean of precision and recall), since it balances catching true job-seekers (recall) against not over-flagging candidates who are staying (precision) — a more meaningful measure than raw accuracy for this imbalanced classification task.

4. Results

4.1 Model Comparison (best threshold per model)

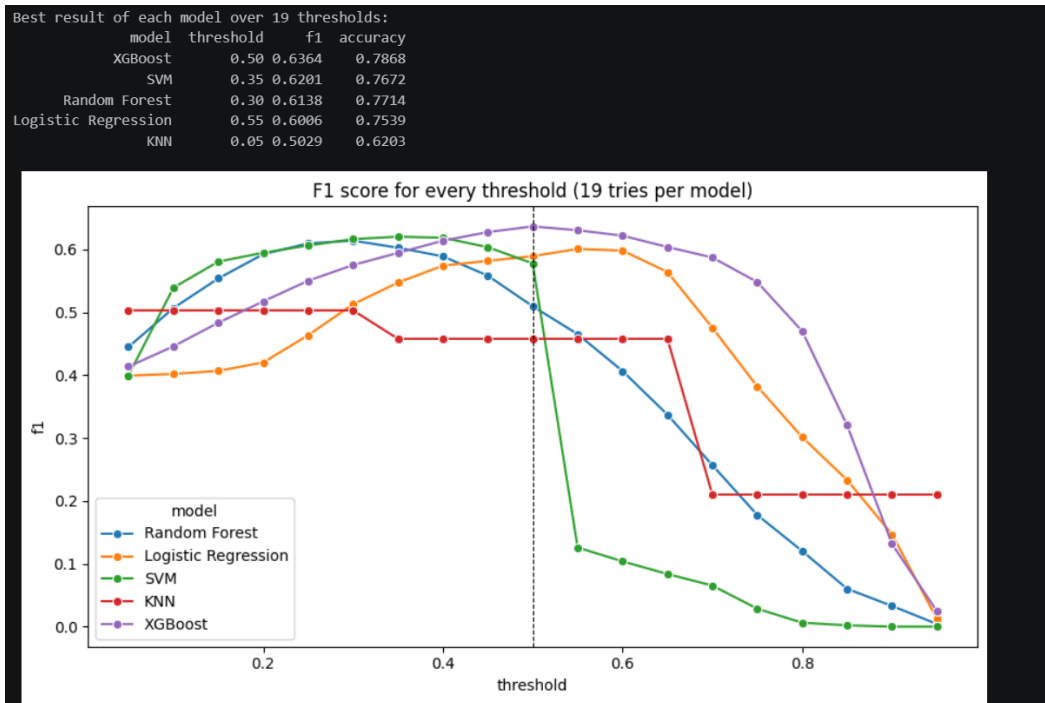


Figure 1 — F1-score across 19 decision thresholds for each of the 5 models.

Model	Best Threshold	F1-Score	Accuracy
XGBoost	0.50	0.6364	0.7868
SVM	0.35	0.6201	0.7672
Random Forest	0.30	0.6138	0.7714
Logistic Regression	0.55	0.6006	0.7539
KNN	0.05	0.5029	0.6203

4.2 Final Model Performance — XGBoost

XGBoost was selected as the final model (threshold = 0.5, F1 = 0.6364).

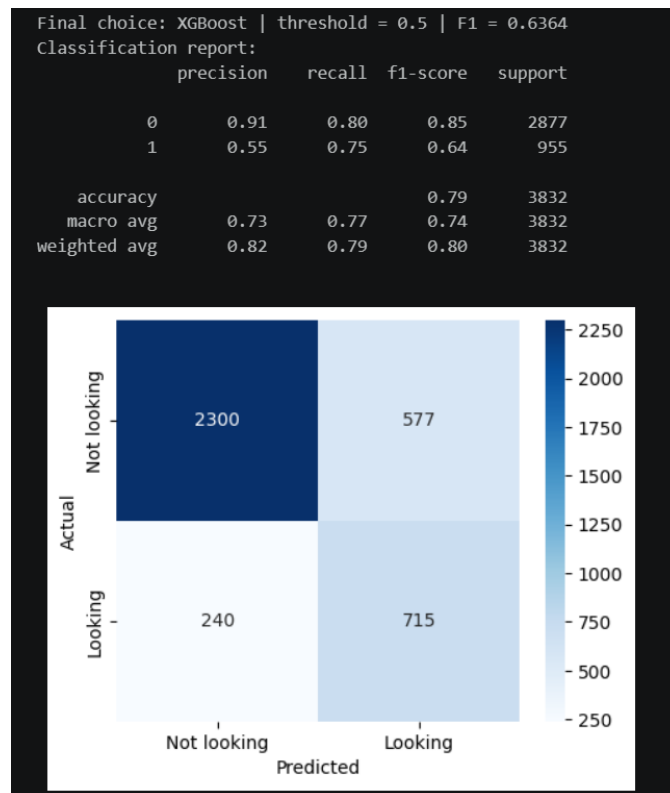


Figure 2 — Classification report and confusion matrix for the final XGBoost model.

Class	Precision	Recall	F1-Score	Support
0 (Not looking)	0.91	0.80	0.85	2,877
1 (Looking)	0.55	0.75	0.64	955
Accuracy			0.79	3,832

The confusion matrix shows the model correctly identifies 715 of 955 actual job-seekers (recall = 0.75), at the cost of 577 false positives among candidates who were not actually looking. This trade-off reflects the threshold choice: prioritizing recall (catching more real job-seekers) is generally preferable for this business use case, since missing an actual job-seeker is more costly than a follow-up conversation with a candidate who was not actually job-hunting.

4.3 Cross-Validation — F1 Reliability Check

Since a single 80/20 split can over- or under-estimate performance, 5-fold stratified cross-validation was run on the XGBoost pipeline, retraining and re-tuning the threshold independently within each fold to avoid information leakage.

Fold	F1-Score
1	0.6118
2	0.6056
3	0.6207
4	0.6164
5	0.6087

Mean F1 across folds: 0.613 ± 0.005 (standard deviation).

The low standard deviation indicates the model's performance is stable across different data splits — the cross-validated F1 (≈ 0.61) is the more defensible number to report, slightly below the single-split estimate of 0.636, which is the expected direction for a single favorable split.

4.4 Candidate Ranking System (Top 10)

Beyond a binary label, candidates were ranked by predicted probability rather than the thresholded class, surfacing the 10 candidates the model is most confident are job-hunting. Ranking quality was validated using Precision@k on the labeled validation set:

k	Precision@k	Correct / k
10	0.60	6 / 10
20	0.60	12 / 20
50	0.66	33 / 50
100	0.61	61 / 100

Precision@k values of 0.60–0.66 are roughly 2.4–2.6x higher than the random-guessing baseline (~ 0.25 , the overall job-seeker rate), confirming the ranking is genuinely informative rather than arbitrary.

5. Conclusions

- XGBoost was the strongest of the five models tested, achieving the highest F1-score (0.636 on a single split; 0.613 ± 0.005 under 5-fold cross-validation) and the best ROC-AUC.
- Class imbalance handling (`class_weight` / `scale_pos_weight`) and per-model threshold tuning were both necessary — the default 0.5 threshold and unweighted classes would have produced a model with near-zero recall on the minority class, as observed in an early unweighted Logistic Regression baseline ($F1 = 0.00$ for class 1).
- An engineered feature capturing 'missing company information' proved to be a meaningful signal, consistent with the hypothesis that these candidates are not currently employed.
- A supplementary PyTorch neural network was tested and underperformed every classical model except KNN ($F1 = 0.591$) — expected on a tabular dataset of this size ($\sim 15K$ training rows), where gradient-boosted trees remain the stronger choice over deep learning.
- The candidate ranking system extends the binary classifier into a practical decision-support tool: rather than a flat list of flagged candidates, HR teams can act on a prioritized Top 10 (or Top-k) list, validated to be 2.4–2.6x more precise than random selection.
- Recommended next steps: hyperparameter tuning beyond the default XGBoost configuration, additional interaction features (e.g. `experience × company_size`), and periodic model refresh as new candidate cohorts are collected.